# Infrastructure Opérateur et Services Clients

## Administration avancée

# Contexte

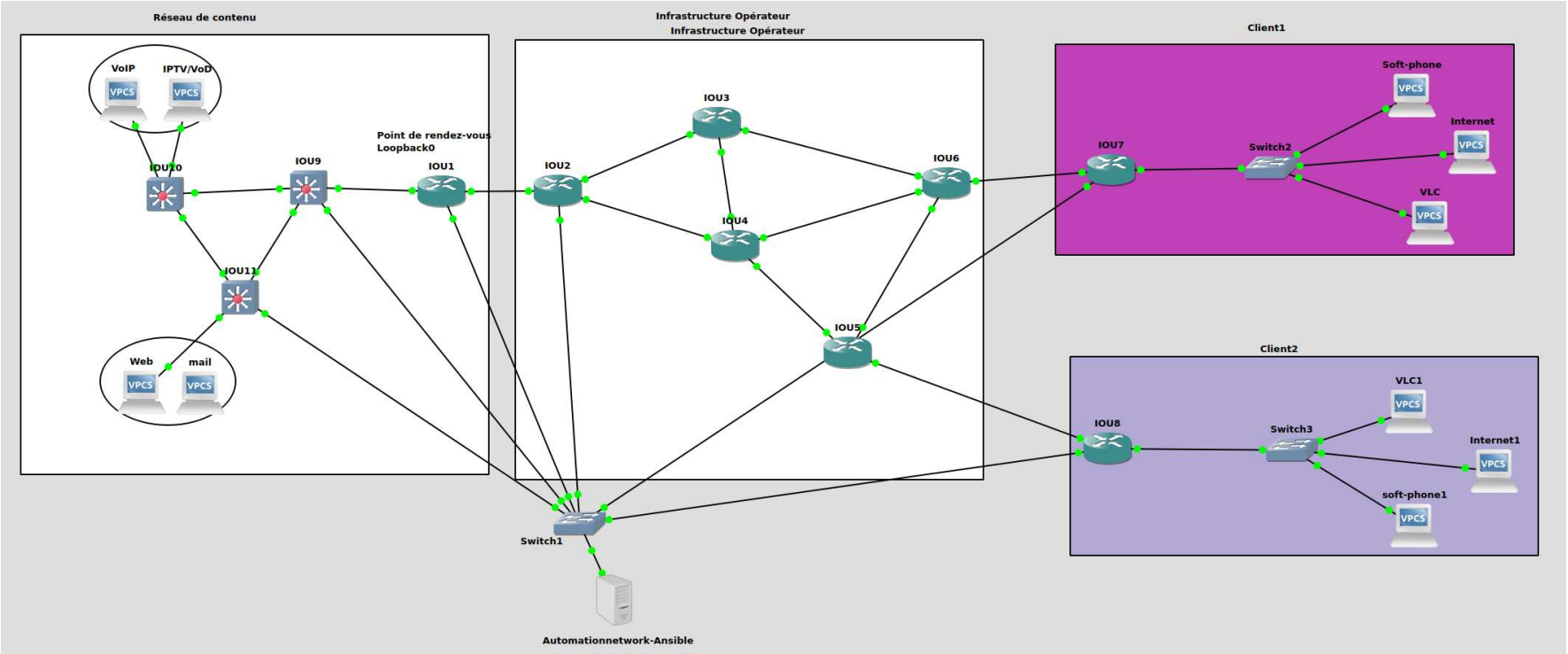
- Concevoir une architecture réseau qui permet de connecter n'importe qui, n'importe où, n'importe quand, sur n'importe quel appareil, de manière sécurisée, fiable et transparente.

- En tant qu'Architecte réseaux :
  - Proposer des solutions en adéquation avec les besoins des cleints : services, ressources, etc.
  - Proposer les bonnes solutions réseaux pour dimensionner l'usage des réseaux : réseaux inter-sites

# Architecture Globale

- Liaison entre deux sites distants via un opérateur internet
- Services réseaux (web, mail, etc.) et services multimédias (VoIP, IPTV, VoD, etc.)
- Fournir une gestion automatisée :
  - Micro-services
  - Haute disponibilité
  - automatisation

# Architecture Globale



# Besoins

- Assurer une communication entre les services:
  - Livraison de contenu
  - Sécurisation entre les sites : DMPVN
  - Diffusion vidéo (IPTV): multicast PIM-SM
  - Communication unicast : routage OSPF
  - Pas de NAT à cause la VoIP
  - Docker pour la gestion des conteneurs
  - Ansible : automatisation réseau
    - VLANs
    - Rouatge

# DIFFUSION DE CONTENUS

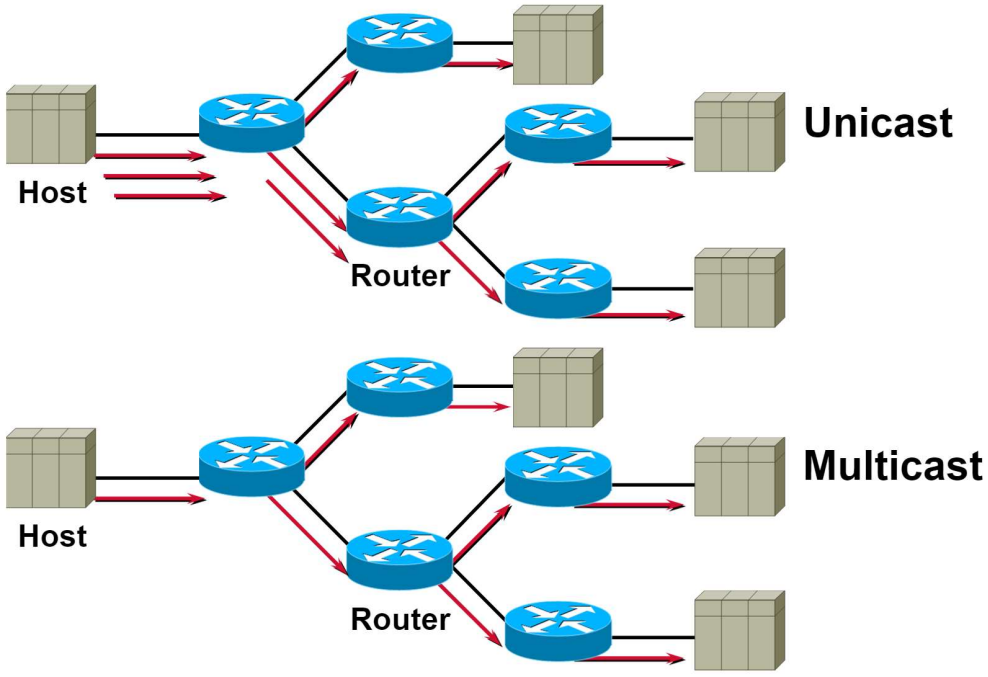
## - Routage multicast : PIM-SM

# INTRODUCTION

- Pourquoi Multicast ?
  - Envoi de données à de multiples récepteurs
  - utilisation optimal de bande passante
  - moins de traitement au niveau routeurs et machines
  - les adresses IP récepteurs sont souvent inconnues
- Applications
  - Vidéo/audio conférences, IPTV, etc.
  - les annonces ou messages de contrôle utilisés par les protocoles réseaux : OSPF, RIPv2

# MULTICAST

- Comment se déroule une communication multicast




- Unicast : envoie multiples de la même data
- Multicast : réplication de la data

# SERVICE MULTICAST

- Les groupes multicast sont identifiés par la classe d'adressage : D
- Les membres du groupe peuvent être présent n'importe où sur internet
- Les membres peuvent joindre/quitter un groupe en informant les routeurs
- Les routeurs écoutent toutes les adresses multicast et utilisent un protocole de routage multicast pour gérer les groupes

# ADRESSES MULTICAST

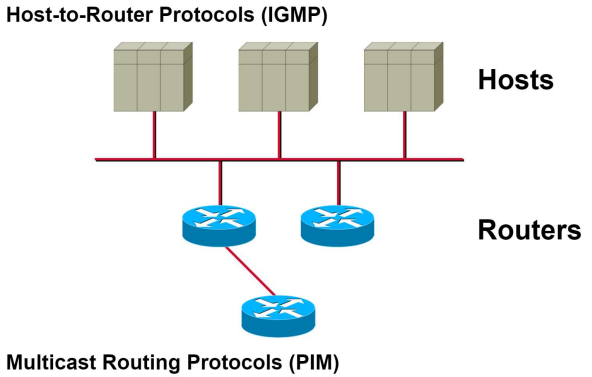
- Classe D

| Cl. | Plage décimale du premier octet | Bits de valeur supérieure du premier octet | Adresse réseau et hôte (R=réseau, H=hôte) | Masque de sous-réseau par défaut | Nombre de réseaux | Hôtes par réseau (adresses utilisables) |
|---|---|---|---|---|---|---|
| **A** | 1 - 126* | 0 | R.H.H.H | 255.0.0.0 | 126 ($2^7$ - 2) | 16,777,214 ($2^{24}$ - 2) |
| **B** | 128 - 191 | 1 0 | R.R.H.H | 255.255.0.0 | 16,382 ($2^{14}$ - 2) | 65,534 ($2^{16}$ - 2) |
| **C** | 192 - 223 | 1 1 0 | R.R.R.H | 255.255.255.0 | 2,097,150 ($2^{21}$ - 2) | 254 ($2^8$ - 2) |
| **D** | 224 - 239 | 1 1 1 0 | Réservée pour la diffusion multicast | | | |
| **E** | 240 - 254 | 1 1 1 1 0 | Expérimentale, utilisée pour la recherche | | | |

- La plage 224.0.0.0 est utilisée par les protocoles de routage : RIPv2, OSPF, IS-IS, etc.

# PROCESSUS D'UNE COMMUNICATION

- Le protocole IGMP (Internet Group Management Protocol) est utilisé pour enregistrer les hôtes dans un groupe de multicast sur un réseau LOCAL.
- Les périphériques de routage multicast utilisent IGMP pour savoir quels groupes ont des membres dans leurs réseaux connectés.
- Le dispositif de routage de multicast peut être un « query ». Un périphérique « Query » envoie des « query »messages pour déterminer si les hôtes sont toujours intéressés par un trafic de groupe de multicast.
- Les hôtes envoient des « report » messages pour confirmer qu'ils sont toujours intéressés par des groupes spécifiques.

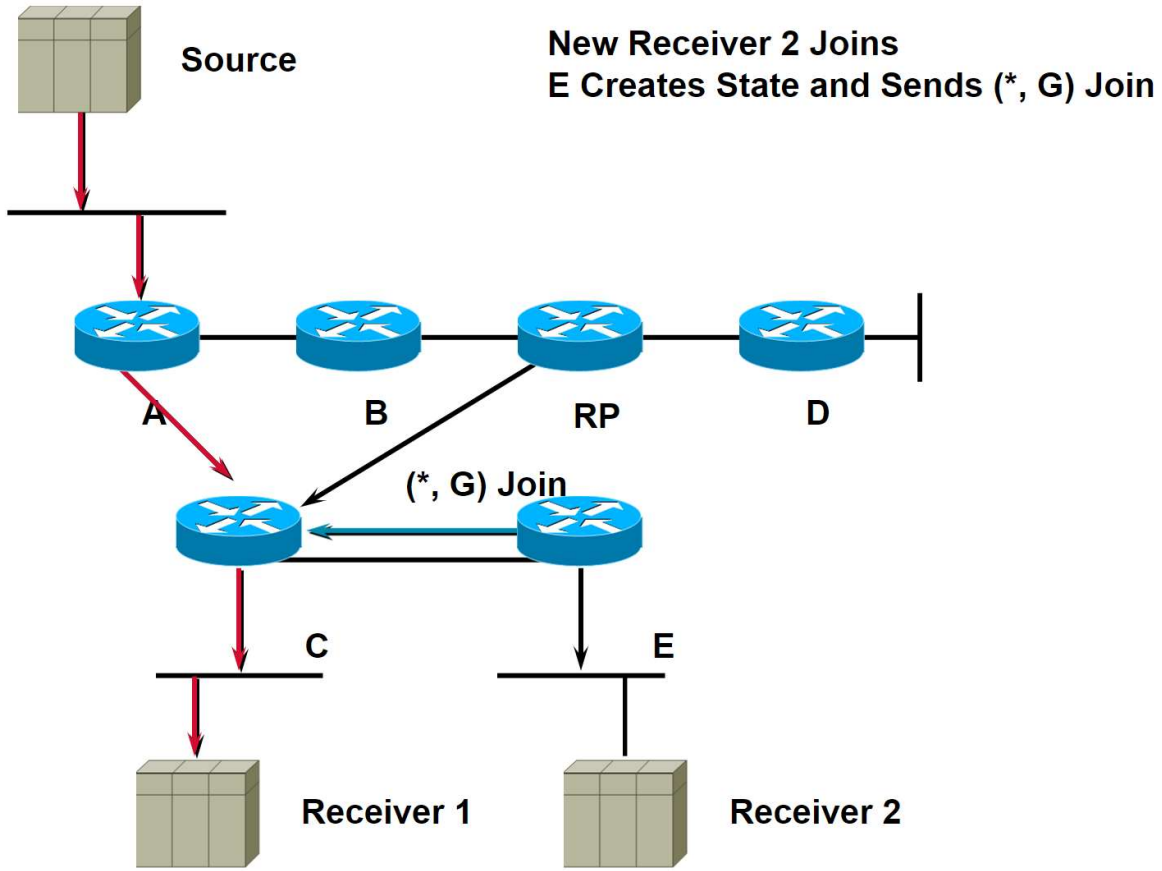


# PROTOCOLE PIM-SM

- Protocol Independent Multicast (PIM) est le protocole de routage de multicast le plus utilisé et permet aux périphériques de routage de multicast de créer ce que l'on appelle **des arbres de distribution**.
- PIM Sparse Mode (PIM-SM) crée l'arborescence partagée en envoyant des messages « join » au RP.
- Un routeur enverra des messages « join » dans ces deux situations:
  - Le routeur reçoit un message « join » sur n'importe quelle interface autre que celle utilisée pour acheminer les paquets vers RP.
  - Le routeur reçoit un message « IGMP report » d'un hôte sur un sous-réseau connecté.

# PROTOCOLE PIM-SM

- Source s'enregistre auprès du RP



# CONFIGURATION

Activer le protocole de routage multicast

```
no ip domain lookup
ip multicast-routing
ip cef
no ipv6 cef
```

- Routeur multicast

```
interface Loopback0
 ip address 1.1.1.1 255.255.255.255
 ip pim sparse-mode
!
interface GigabitEthernet0/0
 ip address 10.1.1.5 255.255.255.0
 ip pim sparse-mode
 duplex auto
 speed auto
 media-type rj45
!
```

```
no ip http server
no ip http secure-server
ip pim rp-address 2.2.2.2
!
```

- Show ip pim rp
- Show ip pim neighbor
- Show ip igmp membership
- Show ip mroute
- show ip igmp groups ~~232.0.1.1~~ detail

# Communications Sécurisées

## - DMVPN

# INTRODUCTION – LES VPNS

- De plus en plus d'entreprises expriment le besoin d'interconnecter leurs différents sites et d'utiliser un moyen de chiffrement afin de protéger leurs communications.

- Le mécanisme IPSec est l'un des solutions le plus utilisé dans la configuration des tunnels VPN pour assurer l'authentification, l'intégrité et la confidentialité des données

# LIMITATIONS DES VPNS IPSEC

- Le VPN IPsec est une solution fiable de communication sécurisée, mais il présente un certain nombre de limites à savoir :

  - Une limite de maintenance et d'échelle : chaque ajout d'un nouveau site conduit à une modification totale de la configuration du site central. La configuration du site central peut facilement devenir illisible au bout d'une dizaine de sites.

  - Pour interconnecter n sites, il faut configurer n(n-1)/2 tunnels et n routeurs ; une équation qui devient difficile à résoudre quand le nombre de sites augmente notamment dans le cas des réseaux Full Meshed ou complètement maillés.

# DMVPN

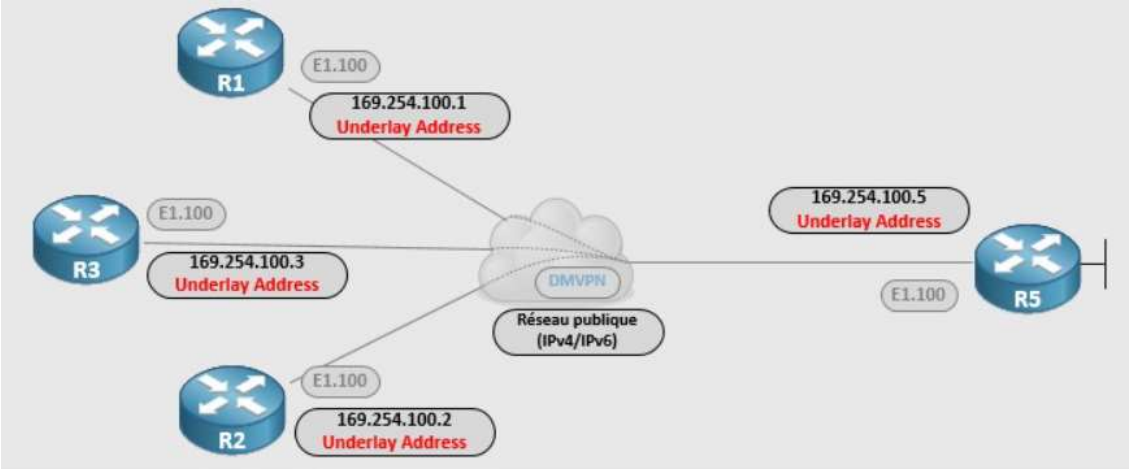
- DMVPN : Dynamic Multi Virtual Private Protocol est une technologie beaucoup plus avantageuse

- Il s'agit d'un mécanisme qui permet d'établir les tunnels IPsec + GRE directement entre les routeurs qui veulent dialoguer de façon totalement dynamique

- Son avantage majeur est qu'il permet de garder la configuration des routeurs statiques en cas d'ajout d'un nouveau site et la création des tunnels entre les sites distants est entièrement automatique.

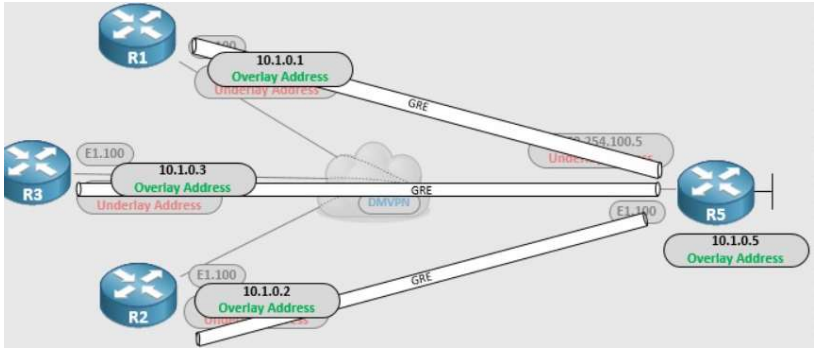
# FONCTIONNEMENT

- Deux IGP sont requis:
  **Underlying** : Pour la connectivité IP publique et monter les tunnels GRE.
  **Overlay**: Pour s'échanger des routes privées une fois le tunnel monté.



# FONCTIONNEMENT – HUB TO SPOKE

- Les Spokes s'enregistrent avec le Hub.
  - Ils spécifient manuellement l'adresse du Hub dans le Tunnel GRE
  - Ils envoient cela via le **NHRP Registration Request**
  - Les Hub apprennent dynamiquement les adresses VPN
  - Les Spokes établissent les tunnels vers les Hub, puis ils échangent ensuite les infos de routage IGP au travers du Tunnel.



# CONFIGURATION

- Configuration du routeur Hub

```
interface Tunnel0
 ip address 192.168.0.1 255.255.255.0
 no ip redirects
 ip nhrp authentication cisco
 ip nhrp map multicast dynamic
 ip nhrp network-id 1
 tunnel source Ethernet0/0
 tunnel mode gre multipoint
 tunnel key 100
```

- Configuration du routeur Spoke

```
interface Tunnel0
 ip address 192.168.0.2 255.255.255.0
 no ip redirects
 ip nhrp authentication cisco
 ip nhrp map multicast dynamic
 ip nhrp map 192.168.0.1 11.0.0.1
 ip nhrp map multicast 11.0.0.1
 ip nhrp network-id 1
 ip nhrp nhs 192.168.0.1
 tunnel source Ethernet0/0
 tunnel mode gre multipoint
 tunnel key 100
```

# Routage Overlay

- Deux méthodes de routage
  - Routage statique en ajoutant les adresses LANs ou route par défaut

```
ip route 0.0.0.0 0.0.0.0 192.168.0.1
```

  - Routage dynamique

```
router rip
 version 2
 network 192.168.0.0
 network 192.168.1.0
```

  - Dans le cas de routage RIP : activer « no ip split-horizon » pour recevoir les « updates de RIP »

# Intégration de la sécurité

- Politique de sécurité

```
crypto isakmp policy 10
 authentication pre-share
crypto isakmp key cisco address 12.0.0.0        255.255.255.0
crypto isakmp key cisco address 13.0.0.0        255.255.255.0
!
!
crypto ipsec transform-set TRANS esp-aes esp-sha-hmac
 mode tunnel
!
!
crypto ipsec profile prof
 set transform-set TRANS
!
```

- Application au tunnel

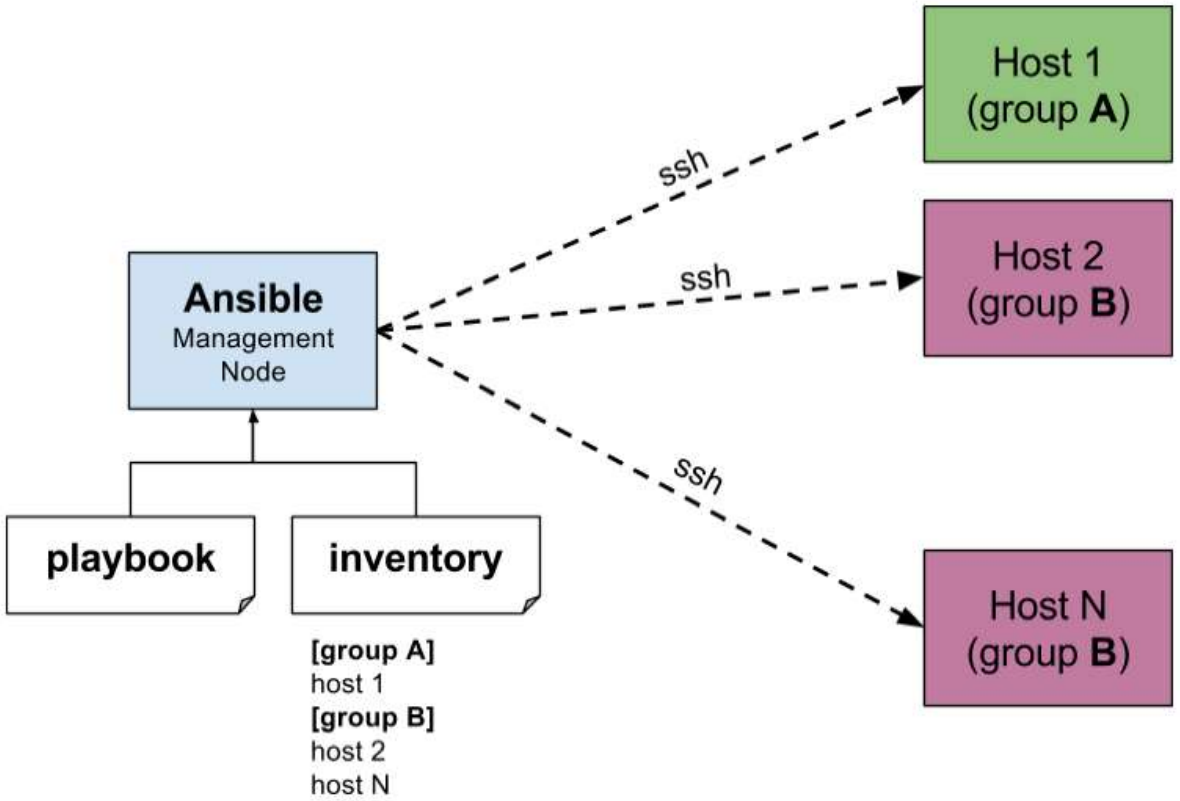
```
tunnel protection ipsec profile prof
```

# Automatisation réseaux - Ansible

# C'EST QUOI ANSIBLE ?

- Ansible est un logiciel libre de gestion des configurations qui automatise le déploiement des applications et la livraison continue des mises à jour.
- Ansible est utilisé pour automatiser 3 types de tâches:
  - Provisionning: configurer les différents équipements dont vous avez besoin dans votre infrastructure.
  - Configuration : modifier la configuration d'une application, ou d'un périphérique, démarrer et arrêter les services, installer ou mettre à jour des applications, mettre en œuvre une politique de sécurité, ou effectuer une grande variété d'autres tâches de configuration.
  - Déploiement d'applications : Automatiser le déploiement d'applications développées en interne sur vos environnements de production.

# Structure de Ansible



# Fichier d'inventaire :

- Le fichier d'inventaire peut être dans l'un des nombreux formats, en fonction des plug-ins d'inventaire dont vous disposez. Les formats les plus courants sont INI et YAML.

```
[switches]
SW1 ansible_ssh_host=10.0.0.10
SW2 ansible_ssh_host=10.0.0.11

[routers]
R1 ansible_ssh_host=10.0.0.12
R2 ansible_ssh_host=10.0.0.13
R4  ansible_ssh_host=10.0.0.14

[network:children]
switches
routers


[all:vars]
ansible_user=toto
ansible_password=toto
ansible_connection=network_cli
ansible_network_os=ios
ansible_become=yes
ansible_become_methode=enable
ansible_become_password=toto
```

# Modules

- Un module est un programme utilisé pour exécuter une tâche ou une commande Ansible.
- Chaque tâche utilise un module et un seul, qui peut prendre des arguments pour être exécuté de manière personnalisée.
- Une tâche est une instruction décrite en YAML dans un fichier de configuration. Chaque tâche utilise un module ainsi que quelques éventuels arguments supplémentaires.
- une tâche Ansible est un appel à un module Ansible accompagné de paramètres. Un module est un programme Ansible en vue de réaliser certaines actions sur les hôtes.

# COMMANDES AD-HOC

- Une commande ad hoc utilise l' outil de ligne de commande /usr/bin/ansible pour automatiser une seule tâche sur un ou plusieurs nœuds gérés.
- Les commandes ad hoc sont idéales pour les tâches que vous répétez rarement.

Exemple: effectuer un ping vers un nœuds

```
root@gns3:/etc/ansible# ansible -m ping network -u toto
SW2 | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
R2 | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
R1 | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
SW1 | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
R4 | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
```

```
root@gns3:/etc/ansible# ansible -m ping R1 -u toto
R1 | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
```

# Commandes Ad-Hoc

- Affichage d'information

```
root@gns3:/etc/ansible# ansible R1 -m ios_command -a "commands='show ip interface brief'"
R1 | SUCCESS => {
    "changed": false,
    "stdout": [
        "Interface              IP-Address      OK? Method Status                Protocol\nFastEthernet0/0        unassigned      YES NVRAM  up
                up      \nFastEthernet0/1        unassigned      YES NVRAM  administratively down down    \nFastEthernet1/0        10.0.0.12
    YES NVRAM  up                    up"
    ],
    "stdout_lines": [
        [
            "Interface              IP-Address      OK? Method Status                Protocol",
            "FastEthernet0/0        unassigned      YES NVRAM  up                    up      ",
            "FastEthernet0/1        unassigned      YES NVRAM  administratively down down    ",
            "FastEthernet1/0        10.0.0.12       YES NVRAM  up                    up"
        ]
    ]
}
```

```
root@gns3:/etc/ansible# ansible R1 -m ios_command -a "commands='show version | include Version'"
R1 | SUCCESS => {
    "changed": false,
    "stdout": [
        "Cisco IOS Software, 7200 Software (C7200-ADVENTERPRISEK9-M), Version 15.2(4)S7, RELEASE SOFTWARE (fc4)\nBOOTLDR: 7200 Software (
VENTERPRISEK9-M), Version 15.2(4)S7, RELEASE SOFTWARE (fc4)\n6 slot VXR midplane, Version 2.1"
    ],
    "stdout_lines": [
        [
            "Cisco IOS Software, 7200 Software (C7200-ADVENTERPRISEK9-M), Version 15.2(4)S7, RELEASE SOFTWARE (fc4)",
            "BOOTLDR: 7200 Software (C7200-ADVENTERPRISEK9-M), Version 15.2(4)S7, RELEASE SOFTWARE (fc4)",
            "6 slot VXR midplane, Version 2.1"
        ]
    ]
}
```

# PLAYBOOKS

- playbook Ansible est un fichier YAML dans lesquels sont mentionnés toutes les tâches qu'Ansible doit exécuter.
- Le fichier YAML commence par décrire les hôtes (ou serveurs ou périphériques) ciblés puis les variables du processus avant de dérouler les jobs à accomplir.
- Les fichiers YAML appellent des modules Ansible pour orchestrer le processus de livraison

# PALYBOOKS

- Structure du fichier :

```yaml
---
- name: Display Hostname and interfaces
  hosts: network
  gather_facts: no
  connection: network_cli
  tasks:
     - name: show commands
       ios_command:
         commands:
         - show run | include hostname
         - show ip int brief
       register: print_result

     - name: print results
       debug:
          var: print_result
```

```
root@gns3:/etc/ansible# ansible-playbook garher_command.yml

PLAY [Display Hostname and interfaces] *****************************************************************************************************************

TASK [show commands] ***********************************************************************************************************************************
ok: [R1]

TASK [print results] ***********************************************************************************************************************************
ok: [R1] => {
    "print_result": {
        "changed": false,
        "failed": false,
        "stdout": [
            "hostname R1",
            "Interface              IP-Address      OK? Method Status                Protocol\nFastEthernet0/0       unassigned      YES NVRAM  up                    up      \nFastEthernet0/1       unassigned      YES NVRAM  administratively down down    \nFastEthernet1/0       10.0.0.1
2       YES NVRAM  up                    up"
        ],
        "stdout_lines": [
            [
                "hostname R1"
            ],
            [
                "Interface              IP-Address      OK? Method Status                Protocol",
                "FastEthernet0/0        unassigned      YES NVRAM  up                    up      ",
                "FastEthernet0/1        unassigned      YES NVRAM  administratively down down    ",
                "FastEthernet1/0        10.0.0.12       YES NVRAM  up                    up"
            ]
        ]
    }
}

PLAY RECAP *********************************************************************************************************************************************
R1                         : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

# Configuration : Routeur

```
---
- name: configure R2
  hosts: R2
  gather_facts: false

  tasks:
    - name: configure f0/0
      ios_config:
        lines:
        - interface f0/0
        - ip address 192.168.3.1 255.255.255.0
        - no shutdown

    - name: configure f0/1
      ios_config:
        lines:
        - interface f0/1
        - ip address 192.168.2.2 255.255.255.0
        - no shutdown
        - exit

    - name: enable OSFP protocol
      ios_config:
        lines:
        - router ospf 1
        - network 192.168.3.0 0.0.0.255 area 0
        - network 192.168.2.0 0.0.0.255 area0
        - do wr mem
```

```
TASK [configure f0/1] ***********************************************************************************************************************************
changed: [R2]

TASK [debug] ********************************************************************************************************************************************
ok: [R2] => {
    "print1": {
        "banners": {},
        "changed": true,
        "commands": [
            "interface f0/1",
            "ip address 192.168.2.2 255.255.255.0",
            "no shutdown",
            "exit"
        ],
        "failed": false,
        "updates": [
            "interface f0/1",
            "ip address 192.168.2.2 255.255.255.0",
            "no shutdown",
            "exit"
        ],
        "warnings": [
            "To ensure idempotency and correct diff the input configuration lines should be similar to how they appear if present in the running
configuration on device"
        ]
    }
}

TASK [enable rip protocol] ******************************************************************************************************************************
changed: [R2]

TASK [debug] ********************************************************************************************************************************************
ok: [R2] => {
    "print": {
        "banners": {},
        "changed": true,
        "commands": [
            "router rip",
            "version 2",
            "network 192.168.3.0",
            "network 192.168.2.0",
            "do wr mem",
```

# CONFIGURATION : SWITCH

```yaml
---
- name: Configure VLAN and assign it to a port on Cisco switch
  hosts: switches
  gather_facts: no
  vars:
    vlan_id: 10
    vlan_name: "VLAN10"
#    interface_name: "GigabitEthernet0/1"
  tasks:
    - name: Create VLAN
      ios_config:
        lines:
          - vlan {{ vlan_id }}
          - name {{ vlan_name }}
      when: inventory_hostname == 'SW1'

    - name: Assign VLAN to interface
      ios_config:
        lines:
          - int range e2/0 - 3
          - switchport mode access
          - switchport access vlan {{ vlan_id }}
      when: inventory_hostname == 'SW1'


    - name: Save configuration
      ios_command:
        commands:
          - write memory
```

```
root@gns3:/etc/ansible# ansible-playbook switch.yml

PLAY [Configure VLAN and assign it to a port on Cisco switch] ******************************************************************

TASK [Create VLAN] *************************************************************************************************************
skipping: [SW2]
[WARNING]: To ensure idempotency and correct diff the input configuration lines should be similar to how they appear if present in
the running configuration on device
changed: [SW1]

TASK [Assign VLAN to interface] ************************************************************************************************
skipping: [SW2]
changed: [SW1]

TASK [Save configuration] ******************************************************************************************************
ok: [SW1]
ok: [SW2]

PLAY RECAP *********************************************************************************************************************
SW1                        : ok=3    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
SW2                        : ok=1    changed=0    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
```

# ROUTAGE INETR-VLANS

```yaml
---
- name: Configure Inter-VLAN Routing with Cisco router
  hosts: routers
  gather_facts: no
  connection: network_cli
  tasks:
    - name: Configure VLAN 10
      ios_config:
        lines:
          - interface f0/0.10
          - encapsulation dot1Q 10
          - ip address 192.168.10.1 255.255.255.0
      when: inventory_hostname == 'R1'  # Replace 'router1' with the actual hostname or identifier of your router

    - name: Configure VLAN 20
      ios_config:
        lines:
          - interface f0/0.20
          - encapsulation dot1Q 20
          - ip address 192.168.20.1 255.255.255.0
      when: inventory_hostname == 'R1'

    - name: Enable routing
      ios_config:
        lines:
          - ip routing
      when: inventory_hostname == 'R1'

    - name: Configure routing between VLANs
      ios_config:
        lines:
          - interface fastEthernet0/0
          - no shutdown
      when: inventory_hostname == 'R1'
```